

```
In[*]:= << mGRG`Einstein`
```

Chap. 4 Components, Transformation, and Types of Indices

4.1 Components and Their Values

4.1.1 Integers as Indices

```
In[*]:= Tdefine[MaxwellF, "-ba", PrintAs -> "F"]
```

텐서의 성분 표현에서 contravariant 인덱스는 양의 정수, covariant 인덱스는 음의 정수를 사용한다: (결과적으로 0은 성분 표현의 인덱스가 될 수 없다.)

```
In[*]:= {MaxwellF[-1, -4], MaxwellF[-4, -1], MaxwellF[4, 1]}
```

Out[*]=

```
{F14, F41, F41}
```

시공간을 4차원으로 설정한다. 따라서 텐서 성분의 최대값은 4가 된다.

```
In[*]:= SetDimension[4]
```

벡터 e와 x를 정의한다:

```
In[*]:= Tdefine[e[ua]]; Tdefine[x[ua]]
```

텐서 성분의 값을 지정한다:

```
In[*]:= SetComponents[MaxwellF[1, 4], -e[1]];
SetComponents[MaxwellF[2, 4], -e[2]];
SetComponents[MaxwellF[3, 4], -e[3]];
```

```
In[*]:= SetComponents[e[1], a Sin[t - z]]
```

텐서 성분은 아니지만 물리학에서 보통 사용하는 표기법인 좌표계의 인덱스 성분이다:

```
In[*]:= SetComponents[x[ua], {x, y, z, t}]
```

$\partial_z F^{14}$ 를 구한다:

```
In[*]:= BD[-3, MaxwellF[1, 4]]
```

Out[*]=

```
-a  $\partial_3$  t Cos[t - z] + a  $\partial_3$  z Cos[t - z] -  $\partial_3$  a Sin[t - z]
```

좌표계를 설정하고 그 좌표계를 이용하여 보통 도함수의 값을 구하도록 한다:

```
In[*]:= SetCoordinates[{x, y, z, t}];
On[EvaluateBDFlag]
```

```
In[*]:= BD[-3, MaxwellF[1, 4]]
Out[*]=
```

$$a \cos[t - z]$$

```
In[*]:= BD[-3, MaxwellF[2, 4]]
Out[*]=
```

$$-\partial_3 e^2$$

기본 설정으로 복귀한다:

```
In[*]:= Off[EvaluateBDFlag] (* default *)
```

4.1.2 Using Integer and Symbolic Indices

설정된 성분 텐서의 값을 취소한다:

```
In[*]:= ClearComponents[e[1]]
```

Gauss 법칙 $\partial_a F^{ab} = 0$:

```
In[*]:= gaussLaw = BD[la, MaxwellF[4, ua]]
Out[*]=
```

$$\partial_a F^{4a}$$

축약된(Contracted) 인덱스의 성분 합을 구한다:

```
In[*]:= SumDum[gaussLaw]
% // TindexSort
```

```
Out[*]=
```

$$\partial_4 F^{44} + \partial_3 e^3 + \partial_2 e^2 + \partial_1 e^1$$

```
Out[*]=
```

$$\partial_3 e^3 + \partial_2 e^2 + \partial_1 e^1$$

전자기장의 자기장 성분과 메트릭 성분을 설정한다:

```
In[*]:= Tdefine[b[ua]]
```

```
In[*]:= SetComponents[MaxwellF[1, 2], b[3]];
SetComponents[MaxwellF[1, 3], -b[2]];
SetComponents[MaxwellF[2, 3], b[1]];

```

```
In[*]:= metric = {{1, 0, 0, 0}, {0, 1, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, -1}};
SetComponents[Metricg[la, lb], metric]
```

전자기장 F_{ab} 의 성분 표현:

```
In[*]:= Metricg[la, lc] × Metricg[lb, ld] × MaxwellF[uc, ud]
% // SumDum
```

```
Out[*]=
```

$$F^{cd} g_{ac} g_{bd}$$

```
Out[*]=
```

$$\begin{aligned} & F^{44} g_{a4} g_{b4} - e^3 g_{a3} g_{b4} - e^2 g_{a2} g_{b4} - e^1 g_{a1} g_{b4} + e^3 g_{a4} g_{b3} + \\ & F^{33} g_{a3} g_{b3} + b^1 g_{a2} g_{b3} - b^2 g_{a1} g_{b3} + e^2 g_{a4} g_{b2} - b^1 g_{a3} g_{b2} + \\ & F^{22} g_{a2} g_{b2} + b^3 g_{a1} g_{b2} + e^1 g_{a4} g_{b1} + b^2 g_{a3} g_{b1} - b^3 g_{a2} g_{b1} + F^{11} g_{a1} g_{b1} \end{aligned}$$

```
In[*]:= dnF = Table[If[la === lb, 0, %], {la, -1, -4, -1}, {lb, -1, -4, -1}]
```

```
Out[*]=
```

$$\{\{0, b^3, -b^2, e^1\}, \{-b^3, 0, b^1, e^2\}, \{b^2, -b^1, 0, e^3\}, \{-e^1, -e^2, -e^3, 0\}\}$$

```
In[*]:= SetComponents[MaxwellF[la, lb], dnF]
Table[MaxwellF[-i, -j], {i, 4}, {j, 4}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 0 & b^3 & -b^2 & e^1 \\ -b^3 & 0 & b^1 & e^2 \\ b^2 & -b^1 & 0 & e^3 \\ -e^1 & -e^2 & -e^3 & 0 \end{pmatrix}$$

```
In[*]:= {MaxwellF[-1, -4], MaxwellF[-2, -3], MaxwellF[-1, -2]}
```

```
Out[*]=
```

$$\{e^1, b^1, b^3\}$$

전자기장 F_{ab} 의 성분 표현 (다른 방법):

```
In[*]:= F2[la_, lb_] := SumDum[Metricg[la, lc] × Metricg[lb, ld] × MaxwellF[uc, ud]]
```

```
In[*]:= {F2[-1, -4], F2[-2, -3], F2[-1, -2]}
```

```
Out[*]=
```

$$\{e^1, b^1, b^3\}$$

전자기장 방정식:

```
In[*]:= eq1[lb_, ua_, ub_] := BD[lb, MaxwellF[ua, ub]] /; PairIndexQ[lb, ub]
eq1[lb, ua, ub]
```

```
Out[*]=
```

$$\partial_b F^{ab}$$

```
In[*]:= eq2[la_, lb_, lc_] := BD[la, MaxwellF[lb, lc]] +  
      BD[lb, MaxwellF[lc, la]] + BD[lc, MaxwellF[la, lb]];  
eq2[la, lb, lc]
```

```
Out[*]=
```

$$\partial_a F_{bc} + \partial_b F_{ca} + \partial_c F_{ab}$$

```
In[*]:= SumDum[eq1[lb, 1, ub]]  
% // TindexSort
```

```
Out[*]=
```

$$-\partial_4 e^1 - \partial_3 b^2 + \partial_2 b^3 + \partial_1 F^{11}$$

```
Out[*]=
```

$$-\partial_4 e^1 - \partial_3 b^2 + \partial_2 b^3$$

```
In[*]:= SumDum[eq1[lb, 2, ub]]  
% // TindexSort
```

```
Out[*]=
```

$$-\partial_4 e^2 + \partial_3 b^1 + \partial_2 F^{22} - \partial_1 b^3$$

```
Out[*]=
```

$$-\partial_4 e^2 + \partial_3 b^1 - \partial_1 b^3$$

```
In[*]:= SumDum[eq1[lb, 3, ub]]  
% // TindexSort
```

```
Out[*]=
```

$$-\partial_4 e^3 + \partial_3 F^{33} - \partial_2 b^1 + \partial_1 b^2$$

```
Out[*]=
```

$$-\partial_4 e^3 - \partial_2 b^1 + \partial_1 b^2$$

```
In[*]:= SumDum[eq1[lb, 4, ub]]  
% // TindexSort
```

```
Out[*]=
```

$$\partial_4 F^{44} + \partial_3 e^3 + \partial_2 e^2 + \partial_1 e^1$$

```
Out[*]=
```

$$\partial_3 e^3 + \partial_2 e^2 + \partial_1 e^1$$

```
In[*]:= eq2[-1, -2, -3]
```

```
Out[*]=
```

$$\partial_3 b^3 + \partial_2 b^2 + \partial_1 b^1$$

```
In[*]:= eq2[-1, -2, -4]
```

```
Out[*]=
```

$$\partial_4 b^3 - \partial_2 e^1 + \partial_1 e^2$$

전자기장 텐서의 불변량:

```
In[*]:= Off[ResetDummiesFlag] (* 인덱스의 자동 재설정을 끈다 *)

In[*]:= MaxwellF[la, lb] × MaxwellF[ua, ub]
SumDum[%, {la, -4, li}, {lb, -4, lj}]
SumDum[%, {1, 3}]

Out[*]:= Fab Fab

Out[*]:= F4j F4j + Fi4 Fi4 + Fij Fij

Out[*]:= 2 b12 + 2 b22 + 2 b32 - 2 e12 - 2 e22 - 2 e32
```

기본 설정으로 복귀:

```
In[*]:= On[ResetDummiesFlag]

전자기장 Fab에 설정된 성분값을 취소한다:

In[*]:= ClearComponents[MaxwellF[la, lb]]

In[*]:= {MaxwellF[-2, -1], MaxwellF[1, 3]}

Out[*]:= {F21, -b2}

In[*]:= {MaxwellF[-1, -1], MaxwellF[-1, -2]} // TindexSort

Out[*]:= {0, F12}
```

4.1.3 Flat Metric in Cartesian Coordinates

시공간의 메트릭을 민코프스키 메트릭으로 설정한다:

```
In[*]:= SetConstantMetric[{1, 1, 1, -1}]
Table[Metricg[-i, -j], {i, 4}, {j, 4}] // MatrixForm

Out[*]//MatrixForm=
```

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

시공간이 3차원 유클리드 공간이 되도록 메트릭을 설정한다:

```
In[*]:= SetConstantMetric[{1, 1, 1}]
Table[Metricg[-i, -j], {i, 3}, {j, 3}] // MatrixForm
```

```
Out[*]//MatrixForm=
```

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
In[*]:= SetCoordinates[{x, y, z}]
```

```
In[*]:= On[EvaluateBDFlag]
```

```
In[*]:= BD[-1, -3, (x y z)^2] (* ∂1∂3(xyz)2 *)
```

```
Out[*]=
```

$$4 x y^2 z$$

```
In[*]:= Off[EvaluateBDFlag] (* default *)
```

상수 메트릭 설정을 해제한다:

```
In[*]:= ClearConstantMetric[]
```

4.1.4 Calculating the Components of Curvature Tensors

Schwarzschild SpaceTime

```
In[*]:= coSys = {r, θ, ϕ, t};
metric = {{1/1 - 2GM/r, 0, 0, 0},
           {0, r^2, 0, 0}, {0, 0, r^2 Sin[θ]^2, 0}, {0, 0, 0, -1 + 2GM/r}};
```

```
In[*]:= Det[metric] // Simplify
```

```
Out[*]=
```

$$-r^4 \sin[\theta]^2$$

주어진 좌표계와 메트릭으로 텐서의 성분 연산을 위한 준비를 한다:

```
In[*]:= SetCTensor[coSys, metric, RicciCD → True] (* 또는 InitCTensor *)
```

연산 결과값을 표시하자:

```
In[*]:= GammaCD[-1, -1, 1]
```

```
Out[*]=
```

$$\frac{GM}{(2GM - r)r}$$

In[*]:= **RiemannCD**[-1, -2, -1, -2]

Out[*]=

$$\frac{G M}{2 G M - r}$$

In[*]:= **RicciCD**[-1, -1]

Out[*]=

$$0$$

CTensor 설정을 해제한다:

In[*]:= **ClearCTensor**[]

Robertson-Walker SpaceTime

In[*]:= **coSys** = {x, y, z, t};
metric = {{a[t]², 0, 0, 0}, {0, a[t]², 0, 0}, {0, 0, a[t]², 0}, {0, 0, 0, -1}};

In[*]:= **SetCTensor**[coSys, metric, RicciCD → True]

In[*]:= **RiemannCD**[-1, -2, -1, -2]

Out[*]=

$$a[t]^2 a'[t]^2$$

In[*]:= **RicciCD**[-1, -1]

Out[*]=

$$2 a'[t]^2 + a[t] a''[t]$$

In[*]:= **Metricg**[1, ua] × **RicciCD**[-1, la]
 % // **SumDum**

Out[*]=

$$g^{1a} R_{1a}$$

Out[*]=

$$\frac{2 a'[t]^2 + a[t] a''[t]}{a[t]^2}$$

In[*]:= **ScalarCD**[]

Out[*]=

$$\frac{6 (a'[t]^2 + a[t] a''[t])}{a[t]^2}$$

4.1.5 Invariants from Curvature Components

Robertson-Walker 시공간의 곡률 텐서로 구성된 불변량을 계산한다:

```
In[*]:= Metricg[ua, uc] × Metricg[ub, ud] × RicciCD[la, lb] × RicciCD[lc, ld]
ricciSquared = SumDum[%] // Expand // Together
```

```
Out[*]=
```

$$g^{ab} g^{cd} R_{ac} R_{bd}$$

```
Out[*]=
```

$$\frac{12 (a'[t]^4 + a[t] a'[t]^2 a''[t] + a[t]^2 a''[t]^2)}{a[t]^4}$$

```
In[*]:= scalarSquared = ScalarCD[]^2
```

```
Out[*]=
```

$$\frac{36 (a'[t]^2 + a[t] a''[t])^2}{a[t]^4}$$

R^{abcd} 의 성분값을 R_{abcd} 로부터 구한다:

```
In[*]:= Metricg[ua, ue] × Metricg[ub, uf] × Metricg[uc, ug] ×
Metricg[ud, uh] × RiemannCD[le, lf, lg, lh]
Timing[riemannUp = Table[SumDum[%], {ua, 4}, {ub, 4}, {uc, 4}, {ud, 4}];]
```

```
Out[*]=
```

$$g^{ae} g^{bf} g^{cg} g^{dh} R_{efgh}$$

```
Out[*]=
```

```
{0.42339, Null}
```

```
In[*]:= SetComponents[RiemannCD[ua, ub, uc, ud], riemannUp]
```

곡률 불변량을 계산한다:

```
In[*]:= RiemannCD[la, lb, lc, ld] × RiemannCD[ua, ub, uc, ud]
riemannSquared = SumDum[%]
```

```
Out[*]=
```

$$R_{abcd} R^{abcd}$$

```
Out[*]=
```

$$\frac{12 a'[t]^4}{a[t]^4} + \frac{12 a''[t]^2}{a[t]^2}$$

```
In[*]:= Det[metric]
```

```
Out[*]=
```

$$-a[t]^6$$

$\sqrt{-g} (R_{abcd} R^{abcd} + R^2 - 4 R_{ab} R^{ab})$ 계산:

```
In[*]:= gaussBonnet =
  Sqrt[-Det[metric]] (riemannSquared + scalarSquared - 4 ricciSquared) //
  Simplify // PowerExpand
gaussBonnet == 8 D[a'[t]^3, t]
```

```
Out[*]:= 24 a'[t]^2 a''[t]
```

```
Out[*]:= True
```

Robertson-Walker 시공간에 대한 CTensor 설정을 해제한다:

```
In[*]:= ClearCTensor[]
```

4.1.6 Simplification and Series Expansion in Components

성분 연산에서 사용되는 규칙을 CsimpifyRules에 설정할 수 있다. 디폴트는 {}이다. 성분 연산 결과를 단순화하는 함수는 Csimpify이다.

매개 변수 q 에 대한 근사 계산으로 Series Expansion을 한다고 할 때 설정하는 Csimpify:

```
In[*]:= Csimpify[expr_] := Series[Expand[expr /. CsimpifyRules], {q, 0, 2}]
```

무한대 근방에서 $1/r$ 의 3차항까지 점근 연산을 한다고 할 때 설정하는 Csimpify:

```
In[*]:= Csimpify[expr_] :=
  Together[Series[Expand[expr /. CsimpifyRules], {r, Infinity, 3}]]
```

Csimpify의 기본 설정:

```
In[*]:= Csimpify[expr_] := Together[expr /. CsimpifyRules] /. CsimpifyRules
```

4.1.7 Schwarzschild with Alternate Simplification

텐서 성분 연산을 위한 단순화 명령을 설정한다:

```
In[*]:= SetOptions[Together, Trig → False];
Csimpify[expr_] := Together[expr /. CsimpifyRules]
```

구형 좌표계에서 슈바르츠실트 메트릭으로 성분 연산을 위한 초기화 설정을 한다:

```
In[*]:= coSys = {r, θ, ϕ, t};
metric = {{1 / (1 - 2GM/r), 0, 0, 0},
  {0, r^2, 0, 0}, {0, 0, r^2 Sin[θ]^2, 0}, {0, 0, 0, - (1 - 2GM/r)}};
```

```
In[*]:= SetCTensor[coSys, metric, RicciCD → True]
```

초기화 설정 후의 결과값을 출력한다:

```
In[*]:= Table[Metricg[ua, ub], {ua, 4}, {ub, 4}]
Out[*]=
```

$$\left\{ \left\{ \frac{-2GM+r}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{r^2}, 0, 0 \right\}, \left\{ 0, 0, \frac{\text{Csc}[\theta]^2}{r^2}, 0 \right\}, \left\{ 0, 0, 0, \frac{r}{2GM-r} \right\} \right\}$$

```
In[*]:= Table[RicciCD[-i, -j], {i, 4}, {j, 4}] // Simplify
Out[*]=
```

$$\{\{0, 0, 0, 0\}, \{0, 0, 0, 0\}, \{0, 0, 0, 0\}, \{0, 0, 0, 0\}\}$$

```
In[*]:= Table[RiemannCD[-i, -j, -i, -j], {i, 2, 4}, {j, i-1}]
Out[*]=
```

$$\left\{ \left\{ \frac{GM}{2GM-r} \right\}, \left\{ \frac{GM \sin[\theta]^2}{2GM-r}, 2GM r \sin[\theta]^2 \right\}, \right. \\ \left. \left\{ -\frac{2GM}{r^3}, -\frac{GM(2GM-r)}{r^2}, -\frac{GM(2GM-r) \sin[\theta]^2}{r^2} \right\} \right\}$$

리만 텐서의 불변량 $R_{abcd} R^{abcd}$ 를 계산한다:

```
In[*]:= Metricg[ua, ue] × Metricg[ub, uf] × Metricg[uc, ug] ×
Metricg[ud, uh] × RiemannCD[le, lf, lg, lh]
Timing[riemannUp = Table[SumDum[%], {ua, 4}, {ub, 4}, {uc, 4}, {ud, 4}];]
Out[*]=
```

$$g^{ae} g^{bf} g^{cg} g^{dh} R_{efgh}$$

```
Out[*]=
```

$$\{0.428938, \text{Null}\}$$

```
In[*]:= SetComponents[RiemannCD[ua, ub, uc, ud], riemannUp]
In[*]:= RiemannCD[la, lb, lc, ld] × RiemannCD[ua, ub, uc, ud]
SumDum[%] // Simplify
Out[*]=
```

$$R_{abcd} R^{abcd}$$

```
Out[*]=
```

$$\frac{48 G^2 M^2}{r^6}$$

```
In[*]:= ClearCTensor[]
```

4.1.8 Asymptotic Behavior in Schwarzschild

성분 텐서 연산을 위한 단순화 명령을 무한대에서의 점근 근사로 설정한다:

```
In[*]:= SetOptions[Expand, Trig → True];
Cimplify[expr_] :=
  Simplify[Series[Expand[expr /. CimplifyRules], {r, Infinity, 3}]]
```

구형 좌표계에서 슈바르츠실트 메트릭으로 성분 연산을 위한 초기화 설정을 한다:

```
In[*]:= coSys = {r, θ, ϕ, t};
metric = {{1/1 - 2GM/r, 0, 0, 0},
  {0, r^2, 0, 0}, {0, 0, r^2 Sin[θ]^2, 0}, {0, 0, 0, -1 - 2GM/r}};
```

```
In[*]:= SetCTensor[coSys, metric, RicciCD → True]
```

초기화 설정 후의 결과값을 출력한다:

```
In[*]:= Table[Metricg[i, j], {i, 4}, {j, i}]
% // Normal
```

Out[*]=

```
{1 - 2(GM)/r + O[1/r]^4, {0, (1/r)^2 + O[1/r]^4, {0, 0, Csc[θ]^2/r^2 + O[1/r]^4,
{0, 0, 0, -1 - 2(GM)/r - 4(G^2 M^2)/r^2 - 8(G^3 M^3)/r^3 + O[1/r]^4}}
```

Out[*]=

```
{1 - 2GM/r, {0, 1/r^2}, {0, 0, Csc[θ]^2/r^2}, {0, 0, 0, -1 - 8G^3 M^3/r^3 - 4G^2 M^2/r^2 - 2GM/r}}
```

```
In[*]:= Table[RicciCD[-i, -j], {i, 4}, {j, i}]
% // Normal
```

Out[*]=

```
{0[1/r]^4, {0[1/r]^3, 0[1/r]^2}, {0, 0, 0[1/r]^2}, {0, 0, 0, 0[1/r]^4}}
```

Out[*]=

```
{{0}, {0, 0}, {0, 0, 0}, {0, 0, 0, 0}}
```

```
In[*]:= Table[RiemannCD[-i, -j, -i, -j], {i, 2, 4}, {j, i - 1}]
% // Normal
```

```
Out[*]=
```

$$\left\{ \left\{ -\frac{GM}{r} + O\left[\frac{1}{r}\right]^2 \right\}, \left\{ -\frac{GM \sin[\theta]^2}{r} + O\left[\frac{1}{r}\right]^2, 2GM \sin[\theta]^2 r + O\left[\frac{1}{r}\right]^0 \right\}, \right. \\ \left. \left\{ -\frac{2(GM)}{r^3} + O\left[\frac{1}{r}\right]^4, \frac{GM}{r} - \frac{2(G^2 M^2)}{r^2} + O\left[\frac{1}{r}\right]^3, \right. \right. \\ \left. \left. \frac{GM \sin[\theta]^2}{r} - \frac{2(G^2 M^2 \sin[\theta]^2)}{r^2} + O\left[\frac{1}{r}\right]^3 \right\} \right\}$$

```
Out[*]=
```

$$\left\{ \left\{ -\frac{GM}{r} \right\}, \left\{ -\frac{GM \sin[\theta]^2}{r}, 2GM r \sin[\theta]^2 \right\}, \right. \\ \left. \left\{ -\frac{2GM}{r^3}, -\frac{2G^2 M^2}{r^2} + \frac{GM}{r}, -\frac{2G^2 M^2 \sin[\theta]^2}{r^2} + \frac{GM \sin[\theta]^2}{r} \right\} \right\}$$

```
In[*]:= ClearCTensor[]
```

4.1.9 Perturbations of Flat Spacetime

텐서 성분 연산을 위한 단순화 명령을 민코프스키 시공간에 대한 섭동으로 설정한다:

```
In[*]:= SetOptions[Expand, Trig -> False];
Csimply[expr_] :=
  Together[Series[Expand[expr /. CsimplyRules], {ε, 0, 1}]]
```

직교 좌표계에서 민코프스키 시공간의 섭동을 묘사하는 메트릭으로 성분 연산을 위한 초기화 설정을 한다:

$$g_{ab} = \begin{pmatrix} 1 + \epsilon h_{11}[t] & \epsilon h_{12}[t] & \epsilon h_{13}[t] & \epsilon h_{14}[t] \\ \epsilon h_{12}[t] & 1 + \epsilon h_{22}[t] & \epsilon h_{23}[t] & \epsilon h_{24}[t] \\ \epsilon h_{13}[t] & \epsilon h_{23}[t] & 1 + \epsilon h_{33}[t] & \epsilon h_{34}[t] \\ \epsilon h_{14}[t] & \epsilon h_{24}[t] & \epsilon h_{34}[t] & -1 + \epsilon h_{44}[t] \end{pmatrix}$$

```
In[*]:= coSys = {x, y, z, t};
metric = {{1 + ε h11[t], ε h12[t], ε h13[t], ε h14[t]},
  {ε h12[t], 1 + ε h22[t], ε h23[t], ε h24[t]}, {ε h13[t], ε h23[t],
  1 + ε h33[t], ε h34[t]}, {ε h14[t], ε h24[t], ε h34[t], -1 + ε h44[t]}};
```

```
In[*]:= SetCTensor[coSys, metric, RicciCD -> True]
```

초기화 설정 후의 결과값을 출력한다:

```
In[*]:= Table[RicciCD[-i, -j], {i, 4}, {j, i}]
% // Normal
```

```
Out[*]=
```

$$\left\{ \left\{ \frac{1}{2} h_{11}''[t] \in + 0[\epsilon]^2 \right\}, \left\{ \frac{1}{2} h_{12}''[t] \in + 0[\epsilon]^2, \frac{1}{2} h_{22}''[t] \in + 0[\epsilon]^2 \right\}, \right. \\ \left. \left\{ \frac{1}{2} h_{13}''[t] \in + 0[\epsilon]^2, \frac{1}{2} h_{23}''[t] \in + 0[\epsilon]^2, \frac{1}{2} h_{33}''[t] \in + 0[\epsilon]^2 \right\}, \right. \\ \left. \left\{ 0[\epsilon]^2, 0[\epsilon]^2, 0[\epsilon]^2, \frac{1}{2} (-h_{11}''[t] - h_{22}''[t] - h_{33}''[t]) \in + 0[\epsilon]^2 \right\} \right\}$$

```
Out[*]=
```

$$\left\{ \left\{ \frac{1}{2} \in h_{11}''[t] \right\}, \left\{ \frac{1}{2} \in h_{12}''[t], \frac{1}{2} \in h_{22}''[t] \right\}, \right. \\ \left. \left\{ \frac{1}{2} \in h_{13}''[t], \frac{1}{2} \in h_{23}''[t], \frac{1}{2} \in h_{33}''[t] \right\}, \right. \\ \left. \left\{ 0, 0, 0, \frac{1}{2} \in (-h_{11}''[t] - h_{22}''[t] - h_{33}''[t]) \right\} \right\}$$

```
In[*]:= ScalarCD[]
```

```
Out[*]=
```

$$(h_{11}''[t] + h_{22}''[t] + h_{33}''[t]) \in + 0[\epsilon]^2$$

```
In[*]:= Table[RiemannCD[-i, -i - 1, -i, -i - 1], {i, 1, 3}]
% // Normal
```

```
Out[*]=
```

$$\left\{ 0[\epsilon]^2, 0[\epsilon]^2, -\frac{1}{2} h_{33}''[t] \in + 0[\epsilon]^2 \right\}$$

```
Out[*]=
```

$$\left\{ 0, 0, -\frac{1}{2} \in h_{33}''[t] \right\}$$

리치 텐서의 불변량 $R_{ab} R^{ab}$ 를 계산한다:

```
In[*]:= Metricg[ua, uc] × Metricg[ub, ud] × RicciCD[la, lb] × RicciCD[lc, ld]
% // SumDum
```

```
Out[*]=
```

$$g^{ab} g^{cd} R_{ac} R_{bd}$$

```
Out[*]=
```

$$0[\epsilon]^2$$

성분 텐서 연산을 위한 CTensor 설정을 취소한다:

```
In[*]:= ClearCTensor[]
```

4.1.10 Evaluating General Tensor Expressions for Particular Metrics

```
In[*]:= On[EvaluateBDFlag]
```

성분 연산을 할 텐서 표현:

```
In[*]:= 2 CD[la, lb, ScalarCD[]] - 1/2 ScalarCD[]^2 Metricg[la, lb] -
        2 CD[uc, lc, ScalarCD[]] × Metricg[la, lb] + 2 ScalarCD[] × RicciCD[la, lb]
```

```
Out[*]:=
```

$$2 \nabla_a \nabla_b R - 2 \nabla^c \nabla_c R g_{ab} + 2 R_{ab} R - \frac{1}{2} g_{ab} R^2$$

```
In[*]:= % // CDtoBD
```

```
% // CDtoBD
```

```
Out[*]:=
```

$$2 \partial_a \nabla_b R - 2 \nabla_c R \Gamma_{ab}^c - 2 \partial^c \nabla_c R g_{ab} + 2 \nabla_c R \Gamma_d^d{}^c g_{ab} + 2 R_{ab} R - \frac{1}{2} g_{ab} R^2$$

```
Out[*]:=
```

$$2 \partial_a \partial_b R - 2 \partial_c R \Gamma_{ab}^c - 2 \partial^c \partial_c R g_{ab} + 2 \partial_c R \Gamma_d^d{}^c g_{ab} + 2 R_{ab} R - \frac{1}{2} g_{ab} R^2$$

```
In[*]:= PutMetric[%, uc]
```

```
PutMetric[%, ud]
```

```
Out[*]:=
```

$$2 \partial_a \partial_b R - 2 \partial_c R \Gamma_{abd} g^{cd} + 2 \partial_c R \Gamma_{de}^d g_{ab} g^{ce} - 2 \partial_c \partial_d R g_{ab} g^{dc} + 2 R_{ab} R - \frac{1}{2} g_{ab} R^2$$

```
Out[*]:=
```

$$2 \partial_a \partial_b R - 2 \partial_c R \Gamma_{abd} g^{dc} - 2 \partial_c \partial_d R g_{ab} g^{dc} + 2 \partial_c R \Gamma_{def} g_{ab} g^{cf} g^{ed} + 2 R_{ab} R - \frac{1}{2} g_{ab} R^2$$

```
In[*]:= hExpr = SumDum[%, {1, 4}];
```

Robertson-Walker 시공간:

```
In[*]:= coSys = {x, y, z, t};
metric = {{a[t]^2, 0, 0, 0}, {0, a[t]^2, 0, 0}, {0, 0, a[t]^2, 0}, {0, 0, 0, -1}};
```

```
In[*]:= SetCTensor[coSys, metric, RicciCD → True]
```

초기화 설정 후의 결과값을 출력한다:

```
In[*]:= hExpr /. {la → -1, lb → -1} // Simplify
```

```
Out[*]:=
```

$$6 \left(\frac{5 a'[t]^4}{a[t]^2} - \frac{12 a'[t]^2 a''[t]}{a[t]} + a''[t]^2 + 4 a'[t] a^{(3)}[t] + 2 a[t] a^{(4)}[t] \right)$$

```
In[*]:= hExpr /. {la → -2, lb → -2} // Simplify
```

```
Out[*]=
```

$$6 \left(\frac{5 a'[t]^4}{a[t]^2} - \frac{12 a'[t]^2 a''[t]}{a[t]} + a''[t]^2 + 4 a'[t] a^{(3)}[t] + 2 a[t] a^{(4)}[t] \right)$$

```
In[*]:= hExpr /. {la → -2, lb → -1}
```

```
Out[*]=
```

0

```
In[*]:= hExpr /. {la → -4, lb → -4} // Simplify
```

```
Out[*]=
```

$$-\frac{18 \left(-5 a'[t]^4 + 2 a[t] a'[t]^2 a''[t] + a[t]^2 a''[t]^2 + 2 a[t]^2 a'[t] a^{(3)}[t] \right)}{a[t]^4}$$

성분 텐서 연산을 위한 CTensor 설정을 취소한다:

```
In[*]:= ClearCTensor[]
```

4.2 Coordinate Transformation of Tensors

4.2.1 Ttransform: Syntax and Explanation

변환 전의 좌표계를 coSys, 변환 후의 좌표계를 newCoSys라고 하자. 이러한 좌표 변환에 따른 텐서 성분의 변화를 구하려면 Ttransform 함수를 사용한다.

변환 전의 텐서 oldT의 성분을 알면서 변환 후의 텐서 newT의 성분을 구하려면

```
Ttransform[oldT[la/ua,...], newT, coSys, newCoSys]
```

반대로, 변환 후의 텐서 newT의 성분을 알면서 변환 전의 텐서 oldT의 성분을 구하려면

```
Ttransform[oldT, newT[la/ua,...], coSys, newCoSys]
```

4.2.2 Four Examples

4.2.3 Example One: Familiar Calculation

삼차원에서 구형 좌표계로부터 직교 좌표계로 변환할 때 메트릭 g_{ab} 의 성분 변환:

```
In[*]:= coSys = {r, θ, ϕ}; newCoSys = {x, y, z};
```

```
Out[*]=
```

```
SequenceForm[MatrixForm[coSys], "→", MatrixForm[newCoSys]]
```

$$\begin{pmatrix} r \\ \theta \\ \phi \end{pmatrix} \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

```
In[*]:= rightRule = {x → r Sin[θ] Cos[ϕ], y → r Sin[θ] Sin[ϕ], z → r Cos[θ]};
```

```
In[*]:= Tdefine[oldg, "ba"]; Tdefine[newg, "ba"]
```

직교 좌표계에서의 newg_{ab} 성분은 알고 있다:

```
In[*]:= SetComponents[newg[la, lb], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}]
```

구형 좌표계에서의 oldg_{ab} 를 구한다:

```
In[*]:= Ttransform[oldg, newg[la, lb], coSys, newCoSys /. rightRule]
```

결과 출력:

```
In[*]:= Table[oldg[-i, -j], {i, 3}, {j, 3}] // Simplify
```

Out[*]=

```
{ {1, 0, 0}, {0, r^2, 0}, {0, 0, r^2 Sin[θ]^2} }
```

4.2.4 Example Two: Simple Shear

좌표계 coSys 는 물체에 ‘고정된’ 경사 좌표계이고, newCoSys 는 물체 외부의 직교 좌표계이다:

```
In[*]:= coSys = {x, y, z}; newCoSys = {x + K y, y, z};
```

직교 좌표계에서의 메트릭:

```
In[*]:= SetComponents[newg[la, lb], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}]
```

좌표계 ‘ coSys ’에서의 메트릭 oldg_{ab} 를 구한다:

```
In[*]:= Ttransform[oldg, newg[la, lb], coSys, newCoSys]
```

```
In[*]:= Table[oldg[-i, -j], {i, 3}, {j, 3}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} 1 & K & 0 \\ K & 1 + K^2 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

좌표계 ‘ coSys ’에서의 메트릭 oldg^{ab} 를 구한다:

```
In[*]:= SetComponents[newg[ua, ub], {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}];
```

```
In[*]:= Ttransform[oldg, newg[ua, ub], coSys, newCoSys]
```

```
In[*]:= Table[oldg[i, j], {i, 3}, {j, 3}] // MatrixForm
```

Out[*]//MatrixForm=

$$\begin{pmatrix} 1 + K^2 & -K & 0 \\ -K & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

4.2.5 Example Three: Special Relativity

x 방향으로 운동하는 관성 좌표계의 로렌츠 변환:

```
In[*]:= 
$$\gamma = \left(1 - \frac{v^2}{c^2}\right)^{-1/2};$$

coSys = {x, y, z, t}; newCoSys = { $\gamma (x - v t)$ , y, z,  $\gamma \left(t - \frac{v}{c^2} x\right)$ };
```

전자기장 F^{ab} 의 'boost' 좌표계에서의 표현을 구한다:

```
In[*]:= Tdefine[F, "-ba"]; Tdefine[newF, "-ba"]
```

```
In[*]:= Ttransform[F[ua, ub], newF, coSys, newCoSys]
```

```
In[*]:= Table[newF[i, j], {i, 1, 4}, {j, 1, 4}] // TindexSort // Simplify
```

```
Out[*]=
```

$$\left\{ \left\{ 0, \frac{F^{12}}{\sqrt{1 - \frac{v^2}{c^2}}} + \frac{v F^{24}}{\sqrt{1 - \frac{v^2}{c^2}}}, \frac{F^{13}}{\sqrt{1 - \frac{v^2}{c^2}}} + \frac{v F^{34}}{\sqrt{1 - \frac{v^2}{c^2}}}, F^{14} \right\}, \right. \\ \left\{ -\frac{F^{12}}{\sqrt{1 - \frac{v^2}{c^2}}} - \frac{v F^{24}}{\sqrt{1 - \frac{v^2}{c^2}}}, 0, F^{23}, \frac{v F^{12}}{c^2 \sqrt{1 - \frac{v^2}{c^2}}} + \frac{F^{24}}{\sqrt{1 - \frac{v^2}{c^2}}} \right\}, \\ \left\{ -\frac{F^{13}}{\sqrt{1 - \frac{v^2}{c^2}}} - \frac{v F^{34}}{\sqrt{1 - \frac{v^2}{c^2}}}, -F^{23}, 0, \frac{v F^{13}}{c^2 \sqrt{1 - \frac{v^2}{c^2}}} + \frac{F^{34}}{\sqrt{1 - \frac{v^2}{c^2}}} \right\}, \\ \left. \left\{ -F^{14}, -\frac{v F^{12}}{c^2 \sqrt{1 - \frac{v^2}{c^2}}} - \frac{F^{24}}{\sqrt{1 - \frac{v^2}{c^2}}}, -\frac{v F^{13}}{c^2 \sqrt{1 - \frac{v^2}{c^2}}} - \frac{F^{34}}{\sqrt{1 - \frac{v^2}{c^2}}}, 0 \right\} \right\}$$

4.2.6 Example Four: General Relativity -- Kruskal Coordinates

슈바르츠실트 블랙홀에서 구형 좌표계로부터 Kruskal 좌표계로의 변환:

```
In[*]:= coSys = {r, t}; newCoSys = { $\sqrt{\frac{r}{2M} - 1} e^{\frac{r}{4M}} \cosh\left[\frac{t}{4M}\right]$ ,  $\sqrt{\frac{r}{2M} - 1} e^{\frac{r}{4M}} \sinh\left[\frac{t}{4M}\right]$ };
```

```
In[*]:= Tdefine[g, "ba"]; Tdefine[newg, "ba"]
```

구형 좌표계에서 슈바르츠실트 블랙홀의 메트릭 표현은 알고 있다:

```
In[*]:= SetComponents[g[la, lb], {{ $\frac{1}{1 - \frac{2M}{r}}$ , 0}, {0,  $-\left(1 - \frac{2M}{r}\right)$ }}]
```

Kruskal 좌표계에서의 메트릭 $newg_{ab}$ 를 구한다:

```
In[*]:= Ttransform[g[la, lb], newg, coSys, newCoSys];
Table[newg[-i, -j], {i, 2}, {j, 2}]
```

```
Out[*]=
```

$$\left\{ \left\{ \frac{32 e^{-\frac{r}{2M}} M^3}{r}, 0 \right\}, \left\{ 0, -\frac{32 e^{-\frac{r}{2M}} M^3}{r} \right\} \right\}$$

따라서 슈바르츠실트 블랙홀의 사건 지평면 $r_h = 2M$ 은 실제 특이점이 아니다.

4.3 Using More Than One Type of Index

4.3.1 Types of Indices

반대칭 텐서를 정의한다:

```
In[*]:= Tdefine[t, "-ba"]
```

텐서 인덱스의 자동적 조정을 끈다:

```
In[*]:= Off[AutoFlag]
```

축약된(Contracted) 인덱스를 더미 인덱스로 바꾼다 (내부적으로 더미 인덱스는 유일하도록 유지된다):

```
In[*]:= t[la, lb] × t[ua, ub]
% // Dum
```

```
Out[*]=
```

$$t_{ab} t^{ab}$$

```
Out[*]=
```

$$t_{\text{Latin\$11734Latin\$11736}} t^{\text{Latin\$11734Latin\$11736}}$$

더미 인덱스를 정규 인덱스로 재조정한다:

```
In[*]:= % // ResetDummies
```

```
Out[*]=
```

$$t_{ab} t^{ab}$$

반대칭 텐서의 trace는 0이다:

```
In[*]:= t[la, ua]
% // Tsimplify
```

```
Out[*]=
```

$$t_a^a$$

```
Out[*]=
```

$$0$$

mGRG에서 기본적으로 준비된 인덱스는 Latin 소문자로 구성된다:

```
In[*]:= GetIndices[Latin]
```

```
Out[*]=
```

```
{la, lb, lc, ld, le, lf, lg, lh, li, lj, lk, ll, lm, ln, lo, lp, lq,
lr, ls, lt, lu, lv, lw, lx, ly, lz, ua, ub, uc, ud, ue, uf, ug, uh,
ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz}
```

따라서 사용할 수 있는 인덱스는 영문 알파벳의 개수와 같다. 그러나 이것이 충분치 않으면 사용할 수 있는 인덱스를 더할 수 있다:

```
In[*]:= AddIndices[ToString /@ {A, B, C, D, E}, Latin]
```

```
In[*]:= GetIndices[Latin]
```

```
Out[*]=
```

```
{la, lb, lc, ld, le, lf, lg, lh, li, lj, lk, ll, lm, ln,
 lo, lp, lq, lr, ls, lt, lu, lv, lw, lx, ly, lz, lA, lB, lC, lD,
 lE, ua, ub, uc, ud, ue, uf, ug, uh, ui, uj, uk, ul, um, un, uo,
 up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz, uA, uB, uC, uD, uE}
```

```
In[*]:= t[lA, uB]
```

```
Out[*]=
```

$$t_A^B$$

새로운 종류의 인덱스를 정의할 수도 있다:

```
In[*]:= DefKind[Bar, "b" <> # & /@ Alphabet[]];
```

```
GetIndices[Bar]
```

```
Out[*]=
```

```
{lba, lbb, lbc, lbd, lbe, lbf, lbg, lbh, lbi, lbj, lbk, lbl, lbm,
 lbn, lbo, lbp, lbq, lbr, lbs, lbt, lbu, lbv, lbw, lbx, lby, lbz,
 uba, ubb, ubc, ubd, ube, ubf, ubg, ubh, ubi, ubj, ubk, ubl, ubm,
 ubn, ubo, ubp, ubq, ubr, ubv, ubt, ubu, ubv, ubw, ubx, uby, ubz}
```

인덱스의 Kind를 Bundle 개념으로 볼 수도 있다. (수학적으로 ‘정확하게’ 동일하지는 않다.) 한 예를 들면, Latin Kind의 인덱스에 대한 메트릭은 g_{ab} 이고, Bar Kind의 메트릭은 $\eta_{\bar{a}\bar{b}}$ 로 설정할 수 있다.

Bar Kind를 위한 메트릭을 정의한다:

```
In[*]:= DefMetric[gBar, Bar]
```

처음 두 개의 인덱스는 Bar Kind, 나머지 두 개의 인덱스는 Latin Kind인 텐서를 정의한다. 각각의 Kind에서 인덱스는 대칭이다:

```
In[*]:= Tdefine[t22[lba, lbb, lc, ld], "+bacd+abdc"]
```

```
In[*]:= t22[lba, lbb, le, lf]
```

```
Out[*]=
```

$$t_{\bar{a}\bar{b}}^{22\bar{e}\bar{f}}$$

```
In[*]:= t22[lba, uba, le, ue]
```

```
Out[*]=
```

$$t_{\bar{a}}^{22\bar{a}}{}^e_e$$

축약된 인덱스를 더미 인덱스로 바꾼다. 더미 인덱스는 내부적으로 유일하게 유지됨을 기억하자:

```
In[*]:= % // Dum
```

```
Out[*]=
```

```
t22Bar$13618 Latin$13620  
Bar$13618 Latin$13620
```

```
In[*]:= t22[lba, lbb, la, lb] × t22[uba, ubb, ua, ub]
```

```
% // Dum
```

```
Out[*]=
```

```
t22abab t22abab
```

```
Out[*]=
```

```
t22Bar$13630Bar$13632Latin$13626Latin$13628 t22Bar$13630Bar$13632Latin$13626Latin$13628
```

```
In[*]:= % // ResetDummies
```

```
Out[*]=
```

```
t22abab t22abab
```

반대칭 텐서 t_{ab} 와 대칭 텐서 $t_{\bar{a}\bar{b}}^{ab}$ 의 텐서곱으로 Latin Kind의 인덱스가 축약되었으므로 0이다.

```
In[*]:= t[l a, l b] × t22[lba, lbb, ua, ub]
```

```
% // Tsimplify
```

```
Out[*]=
```

```
tab t22abab
```

```
Out[*]=
```

```
0
```

내부적으로 유지되는 더미 인덱스로부터 정규 인덱스로의 자동 변환이 이루어지는 기본 설정으로 돌아 간다:

```
In[*]:= On[AutoFlag] (* default *)
```

```
In[*]:= t22[lba, lbb, ua, ub] × t22[ubb, ubc, la, lc] × t22[uba, lbc, lb, uc] +  
t22[lba, lbb, ua, ub] × t22[uba, ubc, lb, lc] × t22[ubb, lbc, la, uc]  
% // Tsimplify
```

```
Out[*]=
```

```
t22abab t22acbc t22bca c + t22abab t22acb c t22bcac
```

```
Out[*]=
```

```
2 t22abab t22ac c t22bcbc
```

4.3.2 Tests and Commands for Various Types of Indices

새로운 종류의 인덱스를 정의한다:

```
In[*]:= DefKind[Dot, "d" <> # & /@ Alphabet[]]
GetIndices[Dot]
```

```
Out[*]=
```

```
{lda, ldb, ldc, ldd, lde, ldf, ldg, ldh, ldi, ldj, ldk, ldl, ldm,
 ldn, ldo, ldp, ldq, ldr, lds, ldt, ldu, ldv, ldw, ldx, ldy, ldz,
 uda, udb, udc, udd, ude, udf, udg, udh, udi, udj, udk, udl, udm,
 udn, udo, udp, udq, udr, uds, udt, udu, udv, udw, udx, udy, udz}
```

새로운 인덱스에도 적용되는 여러 가지 인덱스 관련 함수들을 사용해 보자:

```
In[*]:= DnIndexQ /@ {la, lba, lda, ua, uba, uda}
UpIndexQ /@ {la, lba, lda, ua, uba, uda}
```

```
Out[*]=
```

```
{True, True, True, False, False, False}
```

```
Out[*]=
```

```
{False, False, False, True, True, True}
```

```
In[*]:= KindIndexQ[Latin][#] & /@ {la, lba, lda, ua, uba, uda}
KindIndexQ[Bar][#] & /@ {la, lba, lda, ua, uba, uda}
KindIndexQ[Dot][#] & /@ {la, lba, lda, ua, uba, uda}
```

```
Out[*]=
```

```
{True, False, False, True, False, False}
```

```
Out[*]=
```

```
{False, True, False, False, True, False}
```

```
Out[*]=
```

```
{False, False, True, False, False, True}
```

```
In[*]:= {PairIndexQ[la, ua], PairIndexQ[lba, uba], PairIndexQ[lda, uda]}
```

```
Out[*]=
```

```
{True, True, True}
```

```
In[*]:= ToUpIndex /@ {la, lba, lda}
ToDnIndex /@ {ua, uba, uda}
```

```
Out[*]=
```

```
{ua, uba, uda}
```

```
Out[*]=
```

```
{la, lba, lda}
```

```
In[*]:= FlipIndex /@ {la, ud, lc, uf, lba, ubd, lbc, ubf}
```

```
Out[*]=
```

```
{ua, ld, uc, lf, uba, lbd, ubc, lbf}
```

함수 FindIndices는 텐서 표현의 인덱스를 추출한다:

```
In[*]:= Tdefine[s[lba, lbb, la, lb]]; Tdefine[t[lba, ldb, la, lb]]
```

```
In[*]:= s[lba, uba, lc, ld] × t[udd, ldd, uc, ue]
```

```
Out[*]=
```

$$s_{\bar{a} \text{ ad}}^{\bar{a}} t_{\dot{a}}^{\dot{a} \text{ ae}}$$

```
In[*]:= indexList = FindIndices[%]
```

```
Out[*]=
```

```
{lba, uba, la, ld, uda, lda, ua, ue}
```

```
In[*]:= Select[indexList, KindIndexQ[Bar]]
Select[indexList, KindIndexQ[Dot]]
Select[indexList, KindIndexQ[Latin]]
```

```
Out[*]=
```

```
{lba, uba}
```

```
Out[*]=
```

```
{uda, lda}
```

```
Out[*]=
```

```
{la, ld, ua, ue}
```

기본 설정으로 복귀한다:

```
In[*]:= SetIndices[Alphabet[], Latin] (* default *)
```

4.3.3 Transformation Rules with Multiple Types of Indices

양-밀스 게이지 이론을 위해 새로운 인덱스 계열을 정의하고, 기본 계열로 설정하자:

```
In[*]:= DefKind[Greek, Alphabet["Greek"]];
SetDefaultKind[Greek];
On[CoordinateBasisFlag[Greek]]
```

양-밀스 게이지장 A_{μ}^a 와 $F_{\mu\nu}^a$, 구조 상수 f_{abc} 를 정의하자:

```
In[*]:= Tdefine[YMA[lμ, ua], "ab", PrintAs → "A"];
YMA[lμ, ua]
```

```
Out[*]=
```

$$A_{\mu}^a$$

```
In[*]:= Tdefine[YMF[lμ, lν, ua], "-bac", PrintAs → "F"];
YMF[lμ, lν, ua]
```

```
Out[*]=
```

$$F_{\mu\nu}^a$$

```
In[*]:= Tdefine[YMf[la, lb, lc], "3-", PrintAs -> "f"];
YMf[la, lb, lc]
```

```
Out[*]=
```

$$f_{abc}$$

Latin 계열을 위한 메트릭을 정의한다:

```
In[*]:= DefMetric[YMg, "ϕ", Latin]
```

구조 상수와 게이지장에 대한 규칙을 설정한다:

```
In[*]:= structProdRule = RuleUnique[YMf[a_, b_, e_] × YMf[c_, d_, f_],
  YMg[a, c] × YMg[b, d] - YMg[a, d] × YMg[b, c], PairIndexQ[e, f]]
```

```
Out[*]=
```

$$f_{a=e} f_{c=f} \rightarrow \text{DumFresh}[-\phi_{ad} \phi_{bc} + \phi_{ac} \phi_{bd}] /; \text{PairIndexQ}[e, f]$$

```
In[*]:= YMf[la, lb, le] × YMf[lc, ld, ue]
% /. structProdRule
```

```
Out[*]=
```

$$f_{abe} f_{cd}^e$$

```
Out[*]=
```

$$-\phi_{ad} \phi_{bc} + \phi_{ac} \phi_{bd}$$

```
In[*]:= fieldRule = RuleUnique[YMF[a_, b_, c_],
  BD[a, YMA[b, c]] - BD[b, YMA[a, c]] + e YMf[lp, lq, c] × YMA[a, up] × YMA[b, uq]]
```

```
Out[*]=
```

$$F_{a=c} \rightarrow \text{DumFresh}[\hat{\partial}_a A_{bc} - \hat{\partial}_b A_{ac} + e A_a^{\text{Latin\$13901}} A_b^{\text{Latin\$13903}} f_{\text{Latin\$13901Latin\$13903c}}]$$

```
In[*]:= YMF[lμ, lv, uc]
% /. fieldRule
```

```
Out[*]=
```

$$F_{\mu\nu}^c$$

```
Out[*]=
```

$$\partial_\mu A_\nu^c - \partial_\nu A_\mu^c + e A_\mu^a A_\nu^b f_{ab}^c$$

게이지장의 불변량:

```
In[*]:= Metricg[uμ, uα] × Metricg[uv, uβ] × YMF[lμ, lv, ua] × YMF[lα, lβ, la]
```

```
Out[*]=
```

$$g^{\alpha\beta} g^{\gamma\delta} F_{\alpha\gamma}^a F_{\beta\delta a}$$

In[*]:= % /. fieldRule

Out[*]=

$$\begin{aligned}
 & -\partial_\alpha A_\beta^a \partial_\gamma A_{\delta a} g^{\alpha\delta} g^{\beta\gamma} + 2 \partial_\alpha A_\beta^a \partial_\gamma A_{\delta a} g^{\alpha\gamma} g^{\beta\delta} - \partial_\alpha A_{\beta a} \partial_\gamma A_\delta^a g^{\gamma\beta} g^{\delta\alpha} - \\
 & e \partial_\alpha A_\beta^a g^{\alpha\delta} g^{\beta\gamma} A_\gamma^b A_\delta^c f_{bca} + e \partial_\alpha A_\beta^a g^{\alpha\gamma} g^{\beta\delta} A_\gamma^b A_\delta^c f_{bca} - e \partial_\alpha A_{\beta a} g^{\gamma\beta} g^{\delta\alpha} A_\gamma^b A_\delta^c f_{bc}^a + \\
 & e \partial_\alpha A_{\beta a} g^{\gamma\alpha} g^{\delta\beta} A_\gamma^b A_\delta^c f_{bc}^a + e^2 g^{\alpha\beta} g^{\gamma\delta} A_\alpha^a A_\beta^b A_\gamma^c A_\delta^d f_{ac}^e f_{bde}
 \end{aligned}$$

In[*]:= % // Absorb

Out[*]=

$$\begin{aligned}
 & -\partial^\alpha A_\beta^a \partial^\beta A_{\alpha a} - \partial^\alpha A_{\beta a} \partial^\beta A_\alpha^a + 2 \partial_\alpha A_\beta^a \partial^\alpha A_{\gamma a} g^{\gamma\beta} + e \partial_\alpha A_\beta^a A^{\alpha b} A^{\beta c} f_{bca} + e \partial_\alpha A_{\beta a} A^{\alpha b} A^{\beta c} f_{bc}^a - \\
 & e \partial_\alpha A_\beta^a A^{\alpha b} A^{\beta c} f_{cba} - e \partial_\alpha A_{\beta a} A^{\alpha b} A^{\beta c} f_{cb}^a + e^2 A_\alpha^a A_\beta^b A^{\alpha c} A^{\beta d} f_{ab}^e f_{cde}
 \end{aligned}$$

In[*]:= % // Tsimplify

Out[*]=

$$\begin{aligned}
 & -2 \partial^\alpha A_{\beta a} \partial^\beta A_\alpha^a + 2 \partial_\alpha A_\beta^a \partial^\alpha A_{\gamma a} g^{\gamma\beta} + 2 e \partial_\alpha A_\beta^a A_\alpha^b A_\beta^c f_{bc}^a + \\
 & 2 e \partial_\alpha A_{\beta a} A_\alpha^b A_\beta^c f_{bca} + e^2 A_{\alpha a} A_{\beta b} A_\alpha^c A_\beta^d f_{ab}^e f_{cde}
 \end{aligned}$$

In[*]:= % /. structProdRule

Out[*]=

$$\begin{aligned}
 & -2 \partial^\alpha A_{\beta a} \partial^\beta A_\alpha^a + 2 \partial_\alpha A_\beta^a \partial^\alpha A_{\gamma a} g^{\gamma\beta} + 2 e \partial_\alpha A_\beta^a A_\alpha^b A_\beta^c f_{bc}^a + \\
 & 2 e \partial_\alpha A_{\beta a} A_\alpha^b A_\beta^c f_{bca} - e^2 A_{\alpha a} A_{\beta b} A_\alpha^c A_\beta^d \phi^{ad} \phi^{bc} + e^2 A_{\alpha a} A_{\beta b} A_\alpha^c A_\beta^d \phi^{ac} \phi^{bd}
 \end{aligned}$$

In[*]:= Absorb[%, YMG]

Out[*]=

$$\begin{aligned}
 & -2 \partial^\alpha A_{\beta a} \partial^\beta A_\alpha^a + 2 \partial_\alpha A_\beta^a \partial^\alpha A_{\gamma a} g^{\gamma\beta} - e^2 A_{\alpha a} A_{\beta b} A_\alpha^c A_\beta^d \phi^{ad} \phi^{bc} + \\
 & e^2 A_{\alpha a} A_{\beta b} A_\alpha^c A_\beta^d \phi^{ac} \phi^{bd} + 2 e \partial_\alpha A_\beta^a A_\alpha^b A_\beta^c f_{bc}^a + 2 e \partial_\alpha A_{\beta a} A_\alpha^b A_\beta^c f_{bca}
 \end{aligned}$$

위의 표현에서 $\partial_\alpha \phi^{ab} = 0$ 이므로 마지막 두 개의 항은 같은 표현이다:

In[*]:= BD[lα, YMA[lβ, ua]] × YMF[ub, uc, la]
 PutMetric[%, ua]
 Absorb[%, YMG]
 % /. {BD[α_, YMG[b_, c_]] => 0}

Out[*]=

$$\partial_\alpha A_\beta^a f_{bc}^a$$

Out[*]=

$$\partial_\alpha \phi^{ad} A_{\beta d} f_{bc}^a + \partial_\alpha A_{\beta a} f_{bc}^d \phi^{da}$$

Out[*]=

$$\partial_\alpha \phi^{ad} A_{\beta d} f_{bc}^a + \partial_\alpha A_{\beta a} f_{bca}$$

Out[*]=

$$\partial_\alpha A_{\beta a} f_{bca}$$

기본 설정으로 복귀한다:

```
In[*]:= SetDefaultKind[Latin] (*default*)
```

4.3.4 Warnings about Using Kdelta

4.3.5 Pauli Spin Matrices

새로운 인덱스 계열을 도입한다:

```
In[*]:= DefKind[Capital, ToUpperCase /@ Alphabet[]]
GetIndices[Capital]
```

```
Out[*]:= {lA, lB, lC, lD, lE, lF, lG, lH, lI, lJ, lK, lL, lM, lN, lO, lP, lQ,
          lR, lS, lT, lU, lV, lW, lX, lY, lZ, uA, uB, uC, uD, uE, uF, uG, uH,
          uI, uJ, uK, uL, uM, uN, uO, uP, uQ, uR, uS, uT, uU, uV, uW, uX, uY, uZ}
```

Pauli 시그마 행렬의 인덱스 표기를 위한 텐서를 정의한다:

```
In[*]:= Tdefine[σ[lA, uB, uI], 3]
σ[lA, uB, uI]
```

```
Out[*]:= σABi
```

3차원 유클리드 공간의 메트릭을 설정한다:

```
In[*]:= SetConstantMetric[{1, 1, 1}]
```

시그마 행렬의 곱셈 공식:

$$\sigma_i \sigma_j = \delta_{ij} + \epsilon_{ijk} \sigma_k$$

```
In[*]:= σRule = RuleUnique[σ[lA_, uB_, ua_] × σ[lB_, uC_, ub_],
  Metricg[ua, ub] × Kdelta[lA, uC] + ε Epsilon[ua, ub, uc] × σ[lA, uC, lc],
  PairIndexQ[lB, uB]]
```

```
Out[*]:= σlA_uB_ua_ σlB_uC_ub_ :=>
  DumFresh[δAC gab + εabLatin$14675 σACLatin$14675] /; PairIndexQ[lB, uB]
```

```
In[*]:= σ[lA, uB, 1] × σ[lB, uC, 1]
% /. σRule
% // TindexSort
```

```
Out[*]:= σAB1 σBC1
```

```
Out[*]:= δAC + ε11a σACa
```

```
Out[*]:= δAC
```

```
In[*]:= σ[1A, uB, 1] × σ[1B, uC, 2]
% /. σRule
% // SumDum
```

```
Out[*]=
```

$$\sigma_A^{B1} \sigma_B^{C2}$$

```
Out[*]=
```

$$\mathbb{1} \in^{12a} \sigma_A^C{}_a$$

```
Out[*]=
```

$$\mathbb{1} \sigma_A^C{}_3$$

```
In[*]:= σ[1A, uB, 2] × σ[1B, uC, 1]
% /. σRule
% // SumDum
```

```
Out[*]=
```

$$\sigma_A^{B2} \sigma_B^{C1}$$

```
Out[*]=
```

$$\mathbb{1} \in^{21a} \sigma_A^C{}_a$$

```
Out[*]=
```

$$-\mathbb{1} \sigma_A^C{}_3$$

σ 행렬 공식:

$$(\vec{c} \cdot \vec{\sigma})(\vec{d} \cdot \vec{\sigma}) = \vec{c} \cdot \vec{d} + i(\vec{c} \times \vec{d}) \cdot \vec{\sigma}$$

```
In[*]:= Tdefine[c, 1]; Tdefine[d, 1]
```

```
In[*]:= c[1a] × σ[1A, uB, ua] × σ[1B, uC, ub] × d[1b]
% /. σRule
% // SumDum
```

```
Out[*]=
```

$$c_a d_b \sigma_A^{Ba} \sigma_B^{Cb}$$

```
Out[*]=
```

$$c_a d_b \delta_A^C g^{ab} + \mathbb{1} c_a d_b \epsilon^{abc} \sigma_A^C{}_c$$

```
Out[*]=
```

$$c_3 d_3 \delta_A^C + c_2 d_2 \delta_A^C + c_1 d_1 \delta_A^C + \mathbb{1} c_1 d_2 \sigma_A^C{}_3 - \\ \mathbb{1} c_2 d_1 \sigma_A^C{}_3 - \mathbb{1} c_1 d_3 \sigma_A^C{}_2 + \mathbb{1} c_3 d_1 \sigma_A^C{}_2 + \mathbb{1} c_2 d_3 \sigma_A^C{}_1 - \mathbb{1} c_3 d_2 \sigma_A^C{}_1$$

```
In[*]:= ClearConstantMetric[] (* default *)
```

4.3.6 Summing Expressions with Several Types of Indices

Pauli 행렬:

```
In[*]:=
σMat[1] = {{0, 1}, {1, 0}};
σMat[2] = {{0, -1}, {1, 0}};
σMat[3] = {{1, 0}, {0, -1}};
```

```
In[*]:=
σ[i_Integer /; Negative[i], j_Integer /; Positive[j], n_Integer] :=
σMat[Abs[n]][[-i, j]]
```

```
In[*]:=
σ[1A, uB, 2] × σ[1B, uC, 3]
SumDum[%, {1B, -1, -2}]
Table[%, {uC, 2}, {1A, -1, -2, -1}]
```

Out[*]=

$$\sigma_A^{B2} \sigma_B^{C3}$$

Out[*]=

$$\sigma_1^{C3} \sigma_A^{12} + \sigma_2^{C3} \sigma_A^{22}$$

Out[*]=

$$\{\{0, 1\}, \{1, 0\}\}$$

```
In[*]:=
σMat[2] . σMat[3]
```

Out[*]=

$$\{\{0, 1\}, \{1, 0\}\}$$

```
In[*]:=
expr = Kdelta[1a, ub] × σ[1A, uB, ua] × σ[1B, uC, lb]
SumDum[%, {1a, -1, -2, -3}, {1B, -1, -2}]
Table[%, {uC, 2}, {1A, -1, -2, -1}] // MatrixForm
```

Out[*]=

$$\sigma_A^{Ba} \sigma_B^C$$

Out[*]=

$$\sigma_1^C \sigma_A^{11} + \sigma_1^C \sigma_A^{12} + \sigma_1^C \sigma_A^{13} + \sigma_2^C \sigma_A^{21} + \sigma_2^C \sigma_A^{22} + \sigma_2^C \sigma_A^{23}$$

Out[*]//MatrixForm=

$$\begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}$$

로렌츠 변환을 위해 민코프스키 시공간의 메트릭을 설정한다:

```
In[*]:=
SetConstantMetric[{1, 1, 1, -1}]
```

새로운 인덱스 계열을 도입한다:

```
In[*]:=
SetDefaultKind[Greek]
```

로렌츠 변환 행렬과 속도 벡터를 정의한다:

```
In[*]:=
Tdefine[Λ[uμ, lν]]; Tdefine[v[ua]]
```

메트릭은 로렌츠 변환에 불변이다:

```
In[*]:= lorentzRule = RuleUnique[ $\Lambda[a_, c_] \times \Lambda[b_, d_] \times \text{Metricg}[e_, f_]$ ,  
Metricg[c, d], (PairIndexQ[a, e] && PairIndexQ[b, f]) ||  
(PairIndexQ[a, f] && PairIndexQ[b, e])]
```

```
Out[*]:=  $g_{e_f} \Lambda_{a_c} \Lambda_{\dots} \Rightarrow \text{DumFresh}[g_{cd}] /; (\text{PairIndexQ}[a, e] \&\& \text{PairIndexQ}[b, f]) ||$   
(PairIndexQ[a, f] && PairIndexQ[b, e])
```

```
In[*]:=  $\Lambda[u\mu, l\rho] \times \Lambda[uv, l\sigma] \times \text{Metricg}[l\mu, l\nu]$   
% /. lorentzRule
```

```
Out[*]:=  $g_{\alpha\beta} \Lambda^\alpha_\rho \Lambda^\beta_\sigma$ 
```

```
Out[*]:=  $g_{\rho\sigma}$ 
```

로렌츠 변환 규칙:

```
In[*]:= vRule =  
RuleUnique[ $\Lambda[a_, -1]$ ,  $v[a] \times \Lambda[1, -1]$ , UpIndexQ[a] && KindIndexQ[Latin][a]]
```

```
Out[*]:=  $\Lambda_{a_1} \Rightarrow \text{DumFresh}[v_a \Lambda^1_1] /; \text{UpIndexQ}[a] \&\& \text{KindIndexQ}[\text{Latin}][a]$ 
```

```
In[*]:= { $\Lambda[ua, -1]$ ,  $\Lambda[u\alpha, -1]$ }  
% /. vRule
```

```
Out[*]:=  $\{\Lambda^a_1, \Lambda^\alpha_1\}$ 
```

```
Out[*]:=  $\{v^a \Lambda^1_1, \Lambda^\alpha_1\}$ 
```

메트릭 규칙:

```
In[*]:= g1Rule = {Metricg[-1, a_ /; KindIndexQ[Latin][a]]  $\Rightarrow$  0,  
Metricg[a_ /; KindIndexQ[Latin][a], -1]  $\Rightarrow$  0}
```

```
Out[*]:=  $\{g_{1a_ /; \text{KindIndexQ}[\text{Latin}][a]} \Rightarrow 0, g_{a_ /; \text{KindIndexQ}[\text{Latin}][a]1} \Rightarrow 0\}$ 
```

```
In[*]:= {Metricg[-1, lb], Metricg[-1, l $\mu$ ] }  
% /. g1Rule
```

```
Out[*]:=  $\{g_{1b}, g_{1\mu}\}$ 
```

```
Out[*]:=  $\{0, g_{1\mu}\}$ 
```

```
In[*]:= g2Rule =  
Metricg[a_ /; KindIndexQ[Latin][a], b_ /; KindIndexQ[Latin][b]]  $\Rightarrow$  Kdelta[a, b]
```

```
Out[*]=  

$$g_{a\_ /; \text{KindIndexQ}[\text{Latin}][a]} \overline{\_ /; \text{KindIndexQ}[\text{Latin}][b]} \mapsto \delta_{ab}$$

```

```
In[*]:= Metricg[la, ub]  
% /. g2Rule
```

```
Out[*]=  

$$g_a^b$$

```

```
Out[*]=  

$$\delta_a^b$$

```

```
In[*]:= gRule = Flatten@{g1Rule, g2Rule};
```

```
In[*]:= expr =  $\Lambda[u\mu, -1] \times \Lambda[uv, -1] \times \text{Metricg}[l\mu, lv]$ 
```

```
Out[*]=  

$$g_{\alpha\beta} \Lambda_1^\alpha \Lambda_1^\beta$$

```

```
In[*]:= expr /. lorentzRule
```

```
Out[*]=  

$$1$$

```

```
In[*]:= SumDum[expr, {lμ, la, -1}, {lv, lb, -1}]  
% /. gRule  
% /. vRule
```

```
Out[*]=  

$$\Lambda_1^{1\ 2} + g_{1a} \Lambda_1^1 \Lambda_1^a + g_{a1} \Lambda_1^1 \Lambda_1^a + g_{ab} \Lambda_1^a \Lambda_1^b$$

```

```
Out[*]=  

$$\Lambda_1^{1\ 2} + \delta_{ab} \Lambda_1^a \Lambda_1^b$$

```

```
Out[*]=  

$$\Lambda_1^{1\ 2} + \delta_{ab} v^a v^b \Lambda_1^{1\ 2}$$

```

```
In[*]:= eq = % == -1
```

```
Out[*]=  

$$\Lambda_1^{1\ 2} + \delta_{ab} v^a v^b \Lambda_1^{1\ 2} == -1$$

```

```
In[*]:= Solve[eq,  $\Lambda[1, -1]$ ]
```

```
Out[*]=  

$$\left\{ \left\{ \Lambda_1^1 \rightarrow -\frac{1}{\sqrt{-1 - \delta_{ab} v^a v^b}} \right\}, \left\{ \Lambda_1^1 \rightarrow \frac{1}{\sqrt{-1 - \delta_{ab} v^a v^b}} \right\} \right\}$$

```

4.4 Manipulating Equations in MathTensor

4.5 Unit Systems, Constants, Sign Conventions

4.6 Electromagnetism: Objects and Rules

4.7 Flags and Miscellaneous Commands or Objects